

データマイニング特論

第7回

線形分類モデルと回帰

スライド35,36,40を修正

本科目シラバスの確認 3（授業計画）

#	日程	内容
第 1 回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第 2 回	4/23（木）	pandasによるデータ操作の基礎
第 3 回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第 4 回	5/7（木）	データの前処理と特徴量エンジニアリング
第 5 回	5/14（木）	分類の基礎 ― k近傍法と決定木
第 6 回	5/21（木）	アンサンブル学習
第 7 回	5/28（木）	線形分類モデルと回帰
第 8 回	6/4（木）	モデル評価・選択と不均衡データ
第 9 回	6/11（木）	クラスタリング手法
第 10 回	6/18（木）	アソシエーション分析と異常検知
第 11 回	6/25（木）	テキストマイニング
第 12 回	7/2（木）	時系列データマイニング
第 13 回	7/9（木）	深層学習の概観と使いどころ
第 14 回	7/16（木）	公平性・倫理と再現性
第 15 回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1

線形と回帰

本日の学習内容を確認めよう



回帰とは

【「回帰」とは、 y を x で説明・予測すること】

回帰 = 「ある変数 (y) を、別の変数 (x) で説明・予測するモデル」

(例1) 身長

y : 身長 (連続値)

x : 体重・年齢・性別 $\Rightarrow$ 数値の予測

(例2) 市区町村のコンビニ店舗数

y : コンビニ数 (カウント)

x : 人口・面積・高齢化率 $\Rightarrow$ 数値の予測

(例3) スーパーホストかどうか

y : SHかどうか (0/1)

x : 評価スコア・レビュー件数 $\Rightarrow$ 数値の予測

線形とは

【「線形」とは — 係数 β が足し算で結合していること】

「線形」とは「 x と y の関係が直線」ではない。
「係数 β の組み合わせ方が線形結合」という意味。

◎ 線形モデル（ β が足し算のみ）

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 \leftarrow \text{これも線形！}$$

x^2 を使っているけど係数 β が足し算で結合されているので線形
→ 一般化線形モデル（GLM）・通常の重回帰はこちら

× 非線形モデル（ β が積・指数に入る）

$$y = \beta_0 \times x_1^{\beta_1} \leftarrow \text{非線形}$$

$$y = \exp(\beta_0 + \beta_1 x + \beta_2 x^2)$$

β が指数や積に入ると係数の推定が複雑になる。
→ ニューラルネット・一般化加法モデル（GAM）

線形回帰の限界

【線形回帰の3つの仮定と現実のデータ】

仮定① 応答変数が正規分布に従う

現実： カウント・二値・価格データは正規分布しない

仮定② 分散が一定（等分散性）

現実： ポアソンでは平均が増えると分散も増える

仮定③ 予測値が連続値を取れる

現実： 確率は0～1、カウントは0以上の整数のみ

これら3つの仮定を緩和した枠組みが「GLM（一般化線形モデル）」

失敗する3つの場面

【普通の回帰が失敗する3つの場面】

場面1 カウントデータ（例：市区町村のコンビニ店舗数）

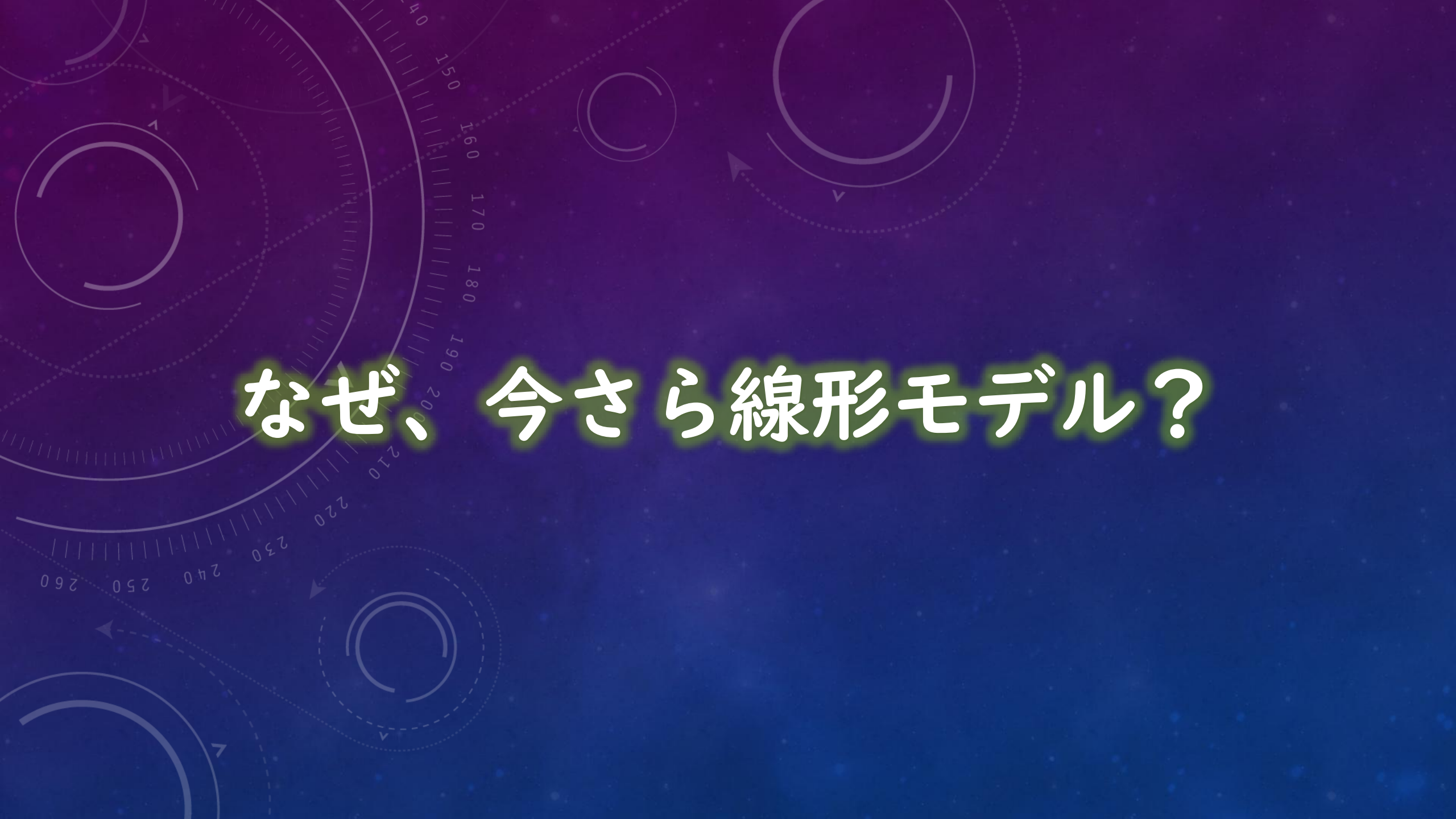
- ⇒負の値は取れない分布が右歪みになる。
- ⇒正規分布を仮定すると負の予測値が出てしまう。
- ⇒そこで、**ポアソン回帰**が有効手段（一般化線形モデルの一種）

場面2 二値データ（0/1）（例：スーパーホストかどうか）

- ⇒0か1しか取れない。
- ⇒線形回帰だと確率が1を超えたり0を下回る。
- ⇒そこで、**ロジスティック回帰**が有効手段（一般化線形モデルの一種）

場面3 正の連続データ（右歪み）（例：宿泊価格・バイオマス）

- ⇒正の値のみ・右に裾が長い。
- ⇒正規分布の仮定が大きく外れる。
- ⇒そこで、**ガンマ回帰**が有効手段（一般化線形モデルの一種）

The background is a dark blue gradient with faint, light blue circular patterns and a degree scale. The scale is a large arc on the left side, with markings from 40 to 260 in increments of 10. Several smaller circles with arrows indicating clockwise or counter-clockwise rotation are scattered across the background.

なぜ、今さら線形モデル？

線形モデルを学ぶ意義（１）

【RFやXGBoostより精度が低いのに、なぜ学ぶ？】

①解釈性が最高クラス

- ⇒係数の符号と大きさから「どの変数が予測にどう効くか」を直接読める。
- ⇒RFの特徴量重要度より詳細。

②計算が圧倒的に速い

- ⇒数百万行のデータでもRF・XGBoostより速い。
- ⇒リアルタイム予測・大規模処理に不可欠。

③正則化で変数選択も同時にできる

- ⇒L1正則化は不要な変数の係数を自動的にゼロにする。
- ⇒前処理不要の変数選択。

④大規模データでの変数選択

- ⇒正則化（L1）は不要な変数の係数を自動的にゼロにする変数選択の機能を持つ。
- ⇒特徴量が多いときの前処理として有効。

線形モデルを学ぶ意義（2）

【線形分類 = 直線（超平面）で境界を引く】

①kNN

⇒曲線的・局所的な境界。

②決定木

⇒縦横の直線的な境界。

③RF/XGBoost

⇒複雑な非線形境界。

④線形モデル

⇒1本の直線（超平面）。

ロジスティック回帰

【ロジスティック回帰 — 確率で語れる線形分類器】

決定木・RFと同じAirbnb Tokyoデータで、ロジスティック回帰を実装して
係数の「方向」を読む。（RFの特徴量重要度にはない情報）

⇒どんな η でもPは必ず0～1に収まる
→ 確率として正しい値が保証される

⇒ロジスティック回帰は「確率を予測する回帰」
→ 0か1かの判定は確率に閾値を設けた後処理

【ロジスティック回帰 — 確率で語れる線形分類器】

出力形式 : 確率 (0～1) を直接予測する

pythonの関数 : `predict_proba()` で確率出力

評価指標 : 精度・F1・ROC-AUC で評価

解釈の強み : 係数の符号で方向が読める (RF重要度より詳細)

本日の環境準備（Ⅰ）

いろいろ試してみよう！

```
# 日本語対応のモジュールをインストールする(その1)
```

```
!pip install japanize-matplotlib
```

```
# 日本語対応のモジュールをインストールする(その2)
```

```
!apt-get install -y fonts-ipafont-gothic
```

```
# 各ライブラリーをインポートする
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.patches as mpatches
```

```
import japanize_matplotlib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.svm import SVC
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.inspection import DecisionBoundaryDisplay
```

```
from sklearn.metrics import (  
    accuracy_score, classification_report,  
    confusion_matrix, ConfusionMatrixDisplay, f1_score,  
    roc_curve, auc
```

```
)
```

本日の環境準備（２）

いろいろ試してみよう！

#データ読み込み

```
URL = "https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/2025-09-29/data/listings.csv.gz"
df = pd.read_csv(URL, compression="gzip", low_memory=False)
print(f"読み込み完了:{df.shape[0]:,} 行 × {df.shape[1]} 列")
```

前処理

```
df["is_superhost"] = (df["host_is_superhost"] == "t").astype(int)
if df["price"].dtype == object:
    df["price"] = df["price"].str.replace(r"[$,]", "", regex=True).astype(float)
```

```
FEATURES = [
    "price", "number_of_reviews", "review_scores_rating",
    "accommodates", "bedrooms", "minimum_nights", "availability_365",
]
```

```
df_clean = df[FEATURES + ["is_superhost"]].dropna()
print(f"前処理後:{df_clean.shape[0]:,} 行")
print(f"スーパーホスト比率:{df_clean['is_superhost'].mean():.1%}")
X = df_clean[FEATURES].values
y = df_clean["is_superhost"].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

線形モデルには標準化が必須(係数の比較のため)

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
print(f"ℳ訓練データ:{len(X_train):,} 件 テストデータ:{len(X_test):,} 件")
```

読み込み完了：27,945 行 × 79 列

前処理後：21,977 行

スーパーホスト比率：44.4%

訓練データ：17,581 件 テストデータ：4,396 件

本日の環境準備（3）

いろいろ試してみよう！

#2次元可視化用データの準備

```
FEAT_X = "review_scores_rating"    # x軸  
FEAT_Y = "number_of_reviews"      # y軸
```

```
X2 = df_clean[[FEAT_X, FEAT_Y]].values  
y2 = df_clean["is_superhost"].values
```

```
X2_train, X2_test, y2_train, y2_test = train_test_split(  
    X2, y2, test_size=0.2, random_state=42, stratify=y2  
)
```

```
scaler2 = StandardScaler()  
X2_train_sc = scaler2.fit_transform(X2_train)  
X2_test_sc = scaler2.transform(X2_test)
```

```
print(f"  x軸:{FEAT_X}")  
print(f"  y軸:{FEAT_Y}")
```

```
x軸: review_scores_rating  
y軸: number_of_reviews
```

ロジスティック回帰 — 構築と係数の解釈 (1)

いろいろ試してみよう！

#ロジスティック回帰 - 構築と係数の解釈

lr = LogisticRegression(C=1.0, random_state=42, max_iter=1000)

lr.fit(X_train_sc, y_train)

y_pred_lr = lr.predict(X_test_sc)

print(f"テスト正解率:{accuracy_score(y_test, y_pred_lr):.3f}")

print(f"SH F1スコア :{f1_score(y_test, y_pred_lr):.3f}")

print(f"SH 再現率 :{classification_report(y_test, y_pred_lr, output_dict=True)['1']['recall']:.3f}")

#次に続く。。。

テスト正解率 : 0.688

SH F1スコア : 0.648

SH 再現率 : 0.648

ロジスティック回帰 — 構築と係数の解釈 (2)

いろいろ試してみよう！

— 係数の可視化(方向と大きさを同時に読む) —

```

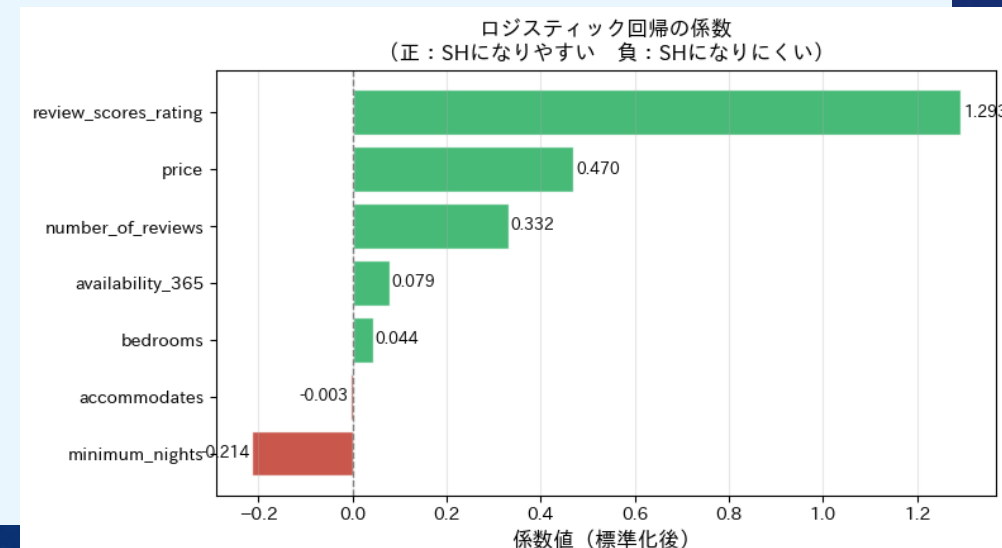
coef_df = pd.DataFrame({
    "feature": FEATURES,
    "coefficient": lr.coef_[0]
}).sort_values("coefficient")

fig, ax = plt.subplots(figsize=(8, 4.5))
colors = ["#C0392B" if c < 0 else "#27AE60"
          for c in coef_df["coefficient"]]
bars = ax.barh(coef_df["feature"], coef_df["coefficient"],
               color=colors, alpha=0.85, edgecolor="white")
ax.axvline(x=0, color="gray", linestyle="--", linewidth=1)

for bar, val in zip(bars, coef_df["coefficient"]):
    ax.text(val + (0.005 if val >= 0 else -0.005),
            bar.get_y() + bar.get_height()/2,
            f"{val:.3f}", va="center",
            ha="left" if val >= 0 else "right", fontsize=9)

ax.set_xlabel("係数値(標準化後)", fontsize=12)
ax.set_title("ロジスティック回帰の係数\n(正:SHになりやすい 負:SHになりにくい)",
            fontsize=12)
ax.grid(True, axis="x", alpha=0.3)
plt.tight_layout()
plt.savefig("lr_coefficients.png", dpi=150, bbox_inches="tight")
plt.show()

```



2

正則化

過学習を防ぐ仕組み



正則化

【係数が大きくなりすぎることにペナルティをかける仕組み】

L2正則化 (Ridge)

- ⇒ $\text{penalty} = \sum \beta_i^2$
- ⇒ すべての係数を均等に縮小、0に近づくが0にはならない
「全員を平等に罰する」
- ⇒ 特徴量がすべて重要な場面
Cが小さいほど強い正則化

L1正則化 (Lasso)

- ⇒ $\text{penalty} = \sum |\beta_i|$
- ⇒ 重要でない係数を完全に0にする、自動的な変数選択が可能
「不要な変数を切り捨てる」
- ⇒ 多くの特徴量のうち一部だけ
重要な場合の変数選択

Cパラメータの影響 (1)

いろいろ試してみよう！

Cパラメータの影響(正則化の強さ)

C_values = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]

results_c = []

for C in C_values:

lr_c = LogisticRegression(C=C, max_iter=1000, random_state=42)

lr_c.fit(X_train_sc, y_train)

pred = lr_c.predict(X_test_sc)

rep = classification_report(pred, y_test, output_dict=True, zero_division=0)

results_c.append({

"C": C,

"テスト正解率": round(accuracy_score(y_test, pred), 3),

"SH再現率": round(rep.get("1", {}).get("recall", 0), 3),

"SH F1": round(f1_score(y_test, pred, zero_division=0), 3),

"非ゼロ係数数": int(np.sum(lr_c.coef_[0] != 0)),

})

c_df = pd.DataFrame(results_c)

print(c_df.to_string(index=False))

#次に続く。。。

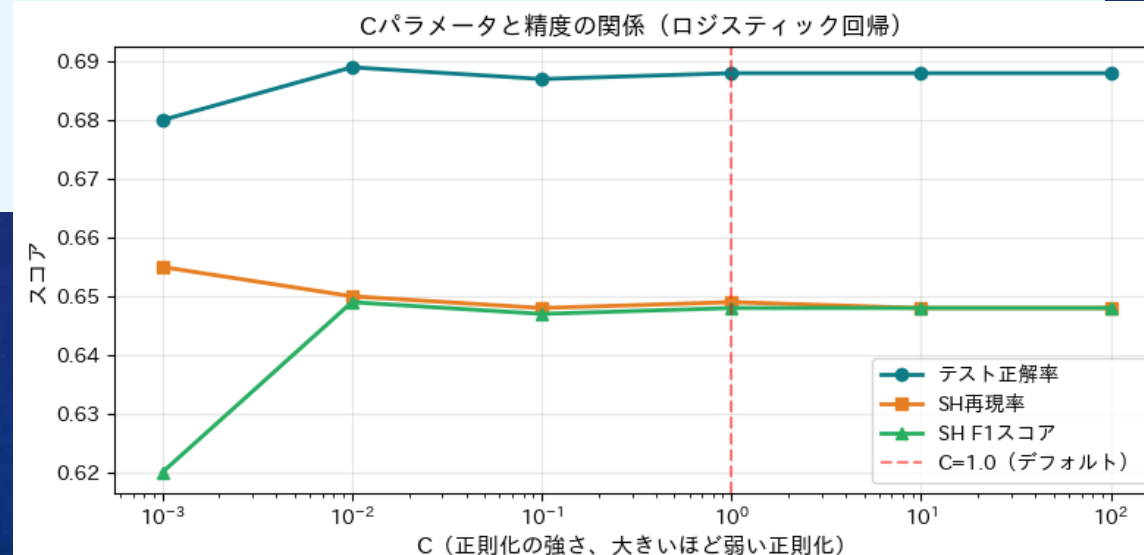
C	テスト正解率	SH再現率	SH F1	非ゼロ係数数
0.001	0.680	0.655	0.620	7
0.010	0.689	0.650	0.649	7
0.100	0.687	0.648	0.647	7
1.000	0.688	0.649	0.648	7
10.000	0.688	0.648	0.648	7
100.000	0.688	0.648	0.648	7

Cパラメータの影響 (2)

いろいろ試してみよう！

C と精度の関係をグラフ化

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogx(c_df["C"], c_df["テスト正解率"], "o-",
            color="#0E7C86", linewidth=2, label="テスト正解率")
ax.semilogx(c_df["C"], c_df["SH再現率"], "s-",
            color="#E67E22", linewidth=2, label="SH再現率")
ax.semilogx(c_df["C"], c_df["SH F1"], "^-",
            color="#27AE60", linewidth=2, label="SH F1スコア")
ax.axvline(x=1.0, color="red", linestyle="--", alpha=0.5,
            label="C=1.0(デフォルト)")
ax.set_xlabel("C(正則化の強さ、大きいほど弱い正則化)", fontsize=11)
ax.set_ylabel("スコア", fontsize=11)
ax.set_title("Cパラメータと精度の関係(ロジスティック回帰)", fontsize=12)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("lr_C_effect.png", dpi=150, bbox_inches="tight")
plt.show()
```



L1 vs L2 正則化の比較 (I)

いろいろ試してみよう！

L1 vs L2 正則化の比較

```
lr_l2 = LogisticRegression(penalty="l2", C=1.0,  
                           max_iter=1000, random_state=42)  
lr_l1 = LogisticRegression(penalty="l1", solver="liblinear",  
                           C=1.0, max_iter=1000, random_state=42)
```

```
lr_l2.fit(X_train_sc, y_train)  
lr_l1.fit(X_train_sc, y_train)
```

```
print("— L2正則化(Ridge)—")  
y_pred_l2 = lr_l2.predict(X_test_sc)  
print(f" テスト正解率:{accuracy_score(y_test, y_pred_l2):.3f}")  
print(f" SH F1スコア :{f1_score(y_test, y_pred_l2):.3f}")  
n_zero_l2 = int(np.sum(lr_l2.coef_[0] == 0))  
print(f" ゼロ係数数 :{n_zero_l2} 個({n_zero_l2}/{len(FEATURES)})")
```

```
print("\n— L1正則化(Lasso)—")  
y_pred_l1 = lr_l1.predict(X_test_sc)  
print(f" テスト正解率:{accuracy_score(y_test, y_pred_l1):.3f}")  
print(f" SH F1スコア :{f1_score(y_test, y_pred_l1):.3f}")  
n_zero_l1 = int(np.sum(lr_l1.coef_[0] == 0))  
print(f" ゼロ係数数 :{n_zero_l1} 個({n_zero_l1}/{len(FEATURES)})")
```

```
if n_zero_l1 > 0:  
    zero_feats = [f for f, c in zip(FEATURES, lr_l1.coef_[0]) if c == 0]  
    print(f" ゼロになった変数:{zero_feats}")
```

#次に続く。。

— L2正則化 (Ridge) —
テスト正解率 : 0.688
SH F1スコア : 0.648
ゼロ係数数 : 0 個 (0/7)

— L1正則化 (Lasso) —
テスト正解率 : 0.688
SH F1スコア : 0.648
ゼロ係数数 : 0 個 (0/7)

L1 vs L2 正則化の比較 (2)

いろいろ試してみよう！

L1とL2の係数を並べて比較

```
coef_compare = pd.DataFrame({  
    "feature": FEATURES,  
    "L2(Ridge)": lr_l2.coef_[0],  
    "L1(Lasso)": lr_l1.coef_[0],  
})  
print("¥n— 係数の比較 —")  
print(coef_compare.round(3).to_string(index=False))
```

#次に続く。。。

— 係数の比較 —

feature	L2 (Ridge)	L1 (Lasso)
price	0.470	0.465
number_of_reviews	0.332	0.332
review_scores_rating	1.293	1.293
accommodates	-0.003	-0.001
bedrooms	0.044	0.043
minimum_nights	-0.214	-0.214
availability_365	0.079	0.079

L1 vs L2 正則化の比較 (3)

いろいろ試してみよう！

グラフで比較

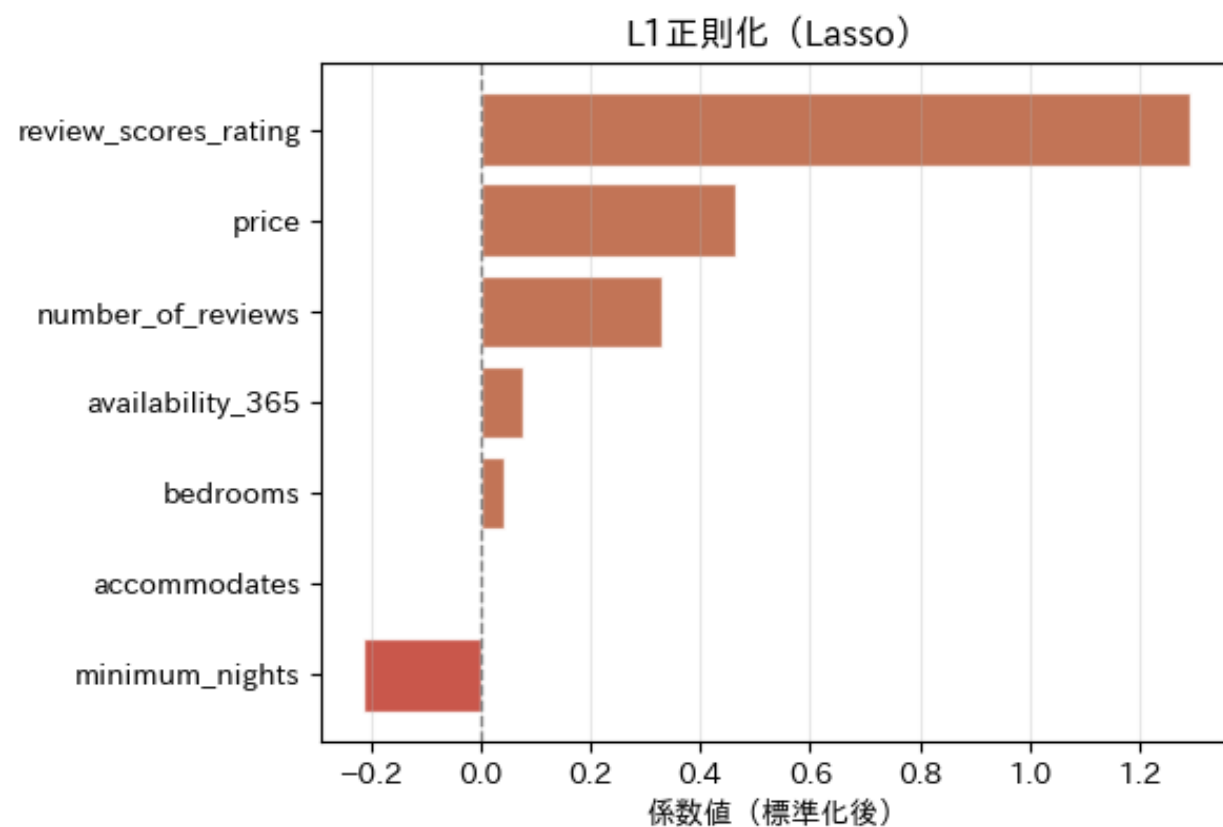
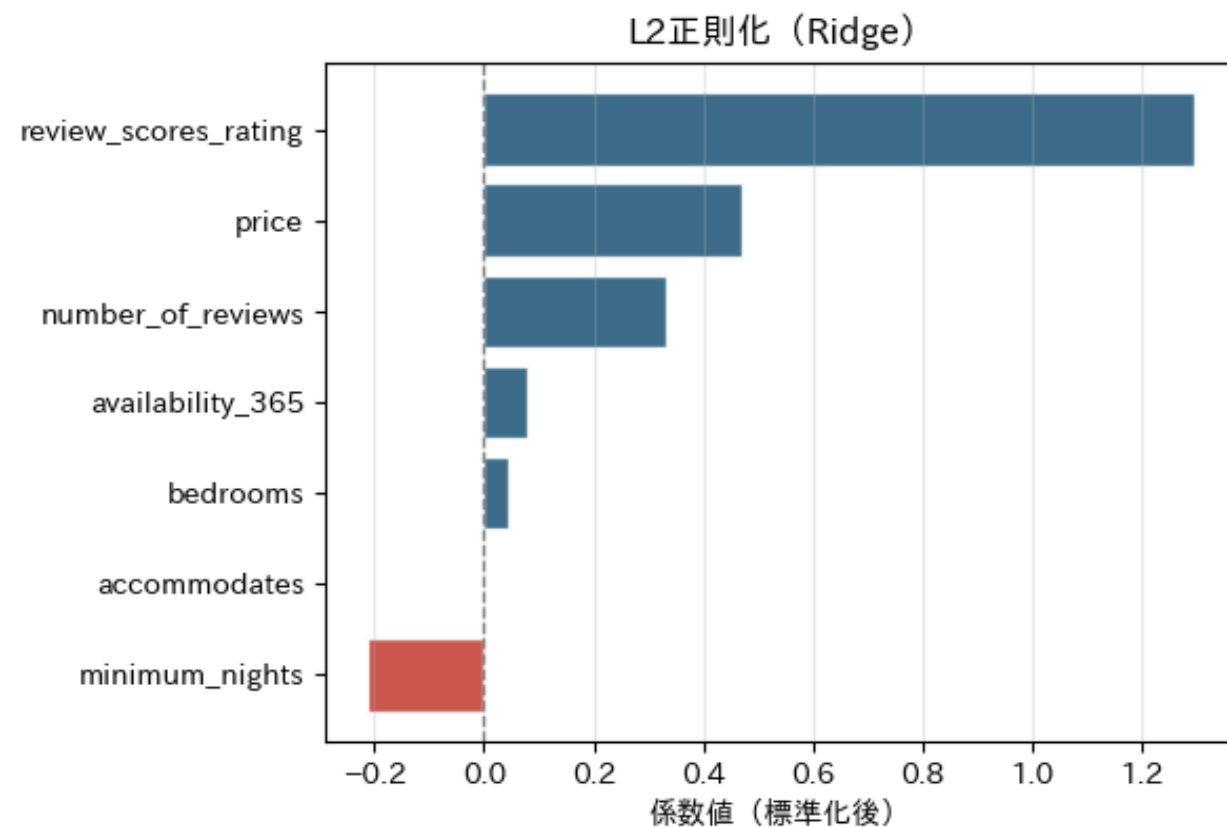
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
for ax, (name, coefs, color) in zip(
    axes,
    [("L2正則化(Ridge)", lr_l2.coef_[0], "#1A5276"),
     ("L1正則化(Lasso)", lr_l1.coef_[0], "#B85C38")]
):
    sorted_idx = np.argsort(coefs)
    feats_sorted = [FEATURES[i] for i in sorted_idx]
    coefs_sorted = coefs[sorted_idx]
    bar_colors = ["#C0392B" if c < 0 else color for c in coefs_sorted]

    ax.barh(feats_sorted, coefs_sorted, color=bar_colors,
            alpha=0.85, edgecolor="white")
    ax.axvline(x=0, color="gray", linestyle="--", linewidth=1)
    ax.set_title(name, fontsize=12, fontweight="bold")
    ax.set_xlabel("係数値(標準化後)", fontsize=10)
    ax.grid(True, axis="x", alpha=0.3)

plt.suptitle("L1 vs L2 正則化:係数の比較(C=1.0)", fontsize=13)
plt.tight_layout()
plt.savefig("l1_vs_l2.png", dpi=150, bbox_inches="tight")
plt.show()
```

L1 vs L2 正則化の比較 (4)

L1 vs L2 正則化 : 係数の比較 (C=1.0)



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on the left side are several concentric circular patterns. A prominent circular scale with degree markings (40, 150, 160, 170, 180, 190, 210, 220, 230, 240, 250, 260) is visible. Other circles of varying sizes and line styles (solid, dashed) are scattered across the left half of the image, some with arrows indicating direction.

サポートベクターマシン

サポートベクターマシン (SVM) (I)

【マージン最大化で境界を引く】

①無数の境界線問題

⇒ 2クラスを分ける直線は無限に引ける。どれが「最良」か？

②マージン最大化

⇒ 両クラスから最も離れた境界線を選ぶ。マージンが広いほど新しいデータに強い。

③サポートベクター

⇒ 境界線の位置を決める少数のデータ点。他のデータ点は境界線の決定に影響しない。

④過学習への耐性

⇒ マージン最大化という原理が自然な正則化として働く。

サポートベクターマシン (SVM) (2)

いろいろ試してみよう！

サポートベクターマシン(SVM)

```
svm = SVC(kernel="linear", C=1.0, random_state=42,  
           probability=True)
```

```
print("SVM学習中(少し時間がかかります)...")
```

```
svm.fit(X_train_sc, y_train)
```

```
y_pred_svm = svm.predict(X_test_sc)
```

```
print(f"¥nテスト正解率:{accuracy_score(y_test, y_pred_svm):.3f}")
```

```
print(f"SH F1スコア :{f1_score(y_test, y_pred_svm):.3f}")
```

```
print(f"SH 再現率 :{classification_report(y_test, y_pred_svm, output_dict=True)['1']['recall']:.3f}")
```

```
print(f"¥nサポートベクターの数(クラス別):{svm.n_support_}")
```

```
print(f" 非SH:{svm.n_support_[0]}個 SH:{svm.n_support_[1]}個")
```

```
print(f" 全体の{sum(svm.n_support_)/len(X_train_sc)*100:.1f}%がサポートベクター")
```

SVM学習中 (少し時間がかかります) ...

テスト正解率 : 0.697

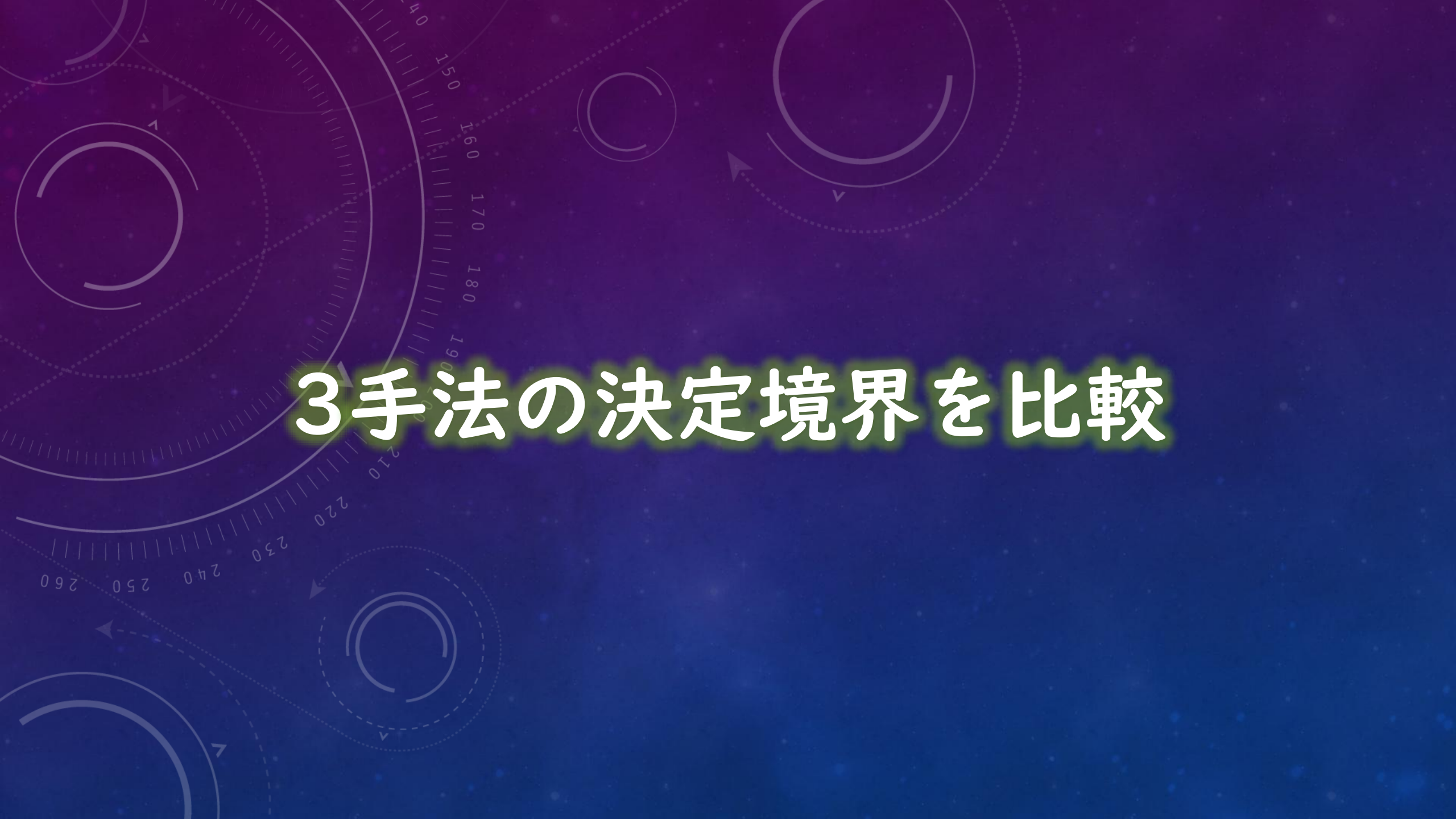
SH F1スコア : 0.682

SH 再現率 : 0.733

サポートベクターの数 (クラス別) : [6374 6379]

非SH : 6374個 SH : 6379個

全体の72.5%がサポートベクター

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale is visible on the left side, with numerical markings from 140 to 260 in increments of 10. Other circular patterns, some with arrows indicating direction, are scattered across the frame. The text is centered and has a soft yellow-green glow.

3手法の決定境界を比較

3手法の決定境界を比較する（1）

【決定境界の比較】

①ロジスティック回帰

⇒ 1本の直線（超平面）
係数が境界の方向を決める

②SVM（線形）

⇒ マージン最大化した直線
最も「安全な」境界

③ランダムフォレスト

⇒ 複雑な非線形境界

3手法の決定境界を比較する (2)

いろいろ試してみよう！

決定境界の比較

2次元データで3手法を再学習(可視化用)

```
lr2 = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
```

```
svm2 = SVC(kernel="linear", C=1.0, random_state=42)
```

```
rf2 = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
```

```
lr2.fit(X2_train_sc, y2_train)
```

```
svm2.fit(X2_train_sc, y2_train)
```

```
rf2.fit(X2_train, y2_train) # RFは標準化不要
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
```

```
configs = [
```

```
    ("ロジスティック回帰¥n(直線の境界)", lr2, X2_train_sc, "#0E7C86"),
```

```
    ("SVM(線形)¥nマージン最大化の境界", svm2, X2_train_sc, "#7D3C98"),
```

```
    ("ランダムフォレスト¥n(複雑な非線形境界)", rf2, X2_train, "#27AE60"),
```

```
]
```

#次に続く。。。

3手法の決定境界を比較する (3)

いろいろ試してみよう！

```
for ax, (title, model, X_2d, accent) in zip(axes, configs):
    DecisionBoundaryDisplay.from_estimator(
        model, X_2d,
        response_method="predict",
        alpha=0.25, cmap="RdYlGn", ax=ax
    )
    ax.scatter(X_2d[:, 0], X_2d[:, 1],
               c=y2_train, cmap="RdYlGn",
               edgecolors="k", linewidths=0.4,
               s=12, alpha=0.6)
    ax.set_title(title, fontsize=11, fontweight="bold")
    ax.set_xlabel(f"{FEAT_X}(標準化後)", fontsize=9)
    ax.set_ylabel(f"{FEAT_Y}(標準化後)", fontsize=9)

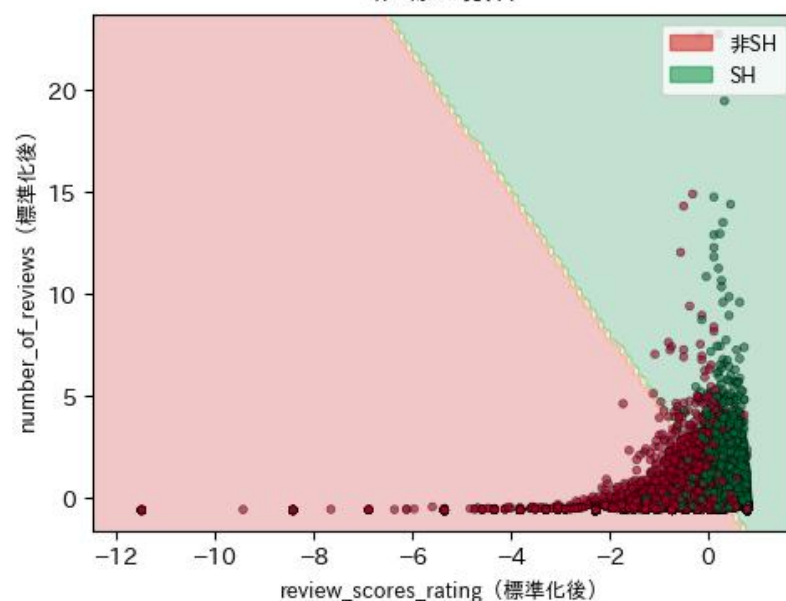
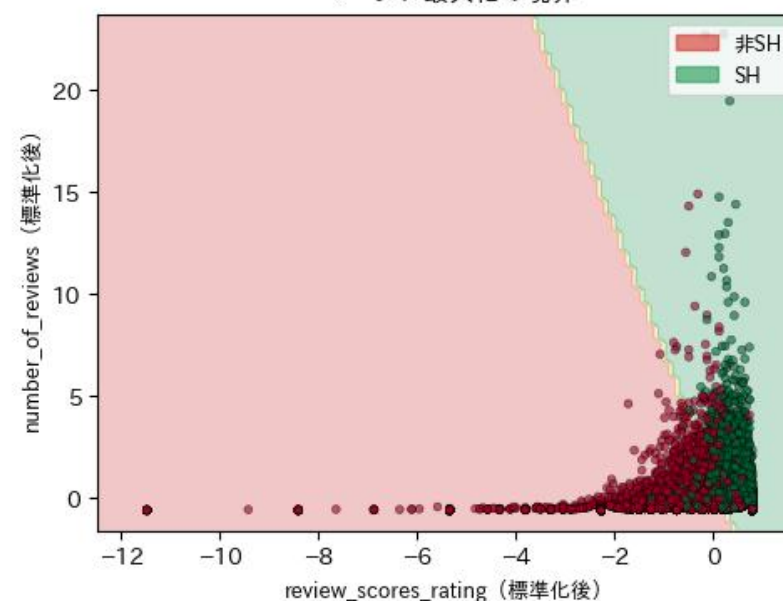
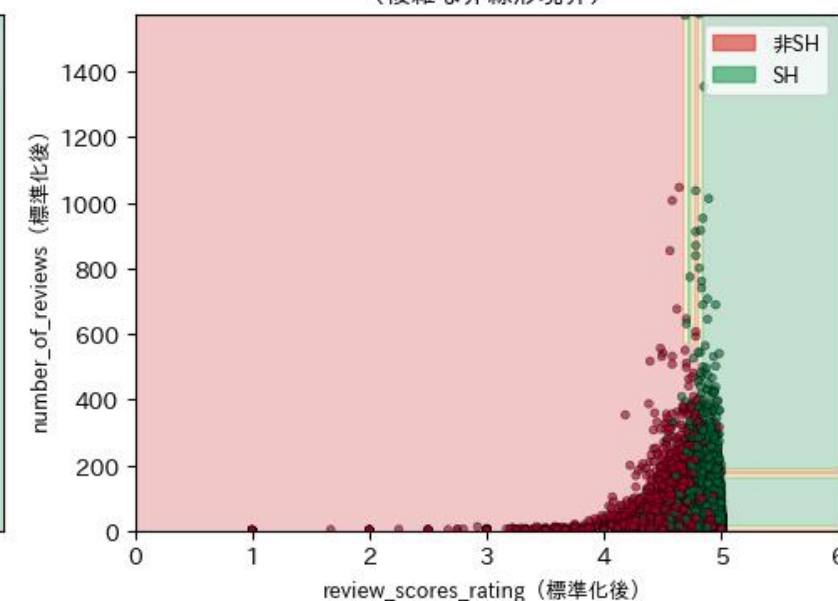
    patch0 = mpatches.Patch(color="#d73027", alpha=0.6, label="非SH")
    patch1 = mpatches.Patch(color="#1a9850", alpha=0.6, label="SH")
    ax.legend(handles=[patch0, patch1], fontsize=9, loc="upper right")

plt.suptitle("決定境界の比較:線形モデル vs ランダムフォレスト",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("decision_boundary_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```

3手法の決定境界を比較する (3)

いろいろ試してみよう!

決定境界の比較：線形モデル vs ランダムフォレスト

ロジスティック回帰
(直線の境界)SVM (線形)
マージン最大化の境界ランダムフォレスト
(複雑な非線形境界)

The background is a gradient from dark purple at the top to dark blue at the bottom, speckled with small white dots. On the left side, there are several concentric circular patterns. One large circle has a degree scale from 140 to 260. Other smaller circles have arrows indicating clockwise or counter-clockwise rotation. The text 'まとめ' is centered in the middle of the image.

まとめ

まとめ

手法	境界の形	解釈性	速度	過学習	主な用途
kNN	曲線的・局所的	×	× (予測遅)	中	プロトタイプ 確認
決定木	縦横の直線	◎ 図で読める	◎	高 (要制御)	説明責任が 必要な場面
ランダムフォレスト (RF)	複雑な非線形	△ 重要度のみ	○	低	精度重視・安定
XGBoost	複雑・精密	△ 重要度のみ	△ (要調整)	要調整	最高精度を狙う
ロジスティック 回帰	直線 (超平面)	◎ 係数±方向	◎◎ 爆速	低 (正則化)	解釈性×速度

第5～7回の全手法比較

いろいろ試してみよう！

第5～6回の手法を再構築

```
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                           random_state=42, n_jobs=-1)
dt.fit(X_train, y_train); y_pred_dt = dt.predict(X_test)
rf.fit(X_train, y_train); y_pred_rf = rf.predict(X_test)
```

XGBoost

import xgboost as xgb

```
xgb_model = xgb.XGBClassifier(
    n_estimators=200, learning_rate=0.1, max_depth=4,
    subsample=0.8, use_label_encoder=False,
    eval_metric="logloss", random_state=42, n_jobs=-1, verbosity=0
)
```

```
xgb_model.fit(X_train, y_train)
y_pred_xgb = xgb_model.predict(X_test)
```

#次に続く。。。

第5～7回の全手法比較

いろいろ試してみよう！

全手法の評価

```
def get_metrics(y_true, y_pred, name):
    rep = classification_report(y_true, y_pred, output_dict=True, zero_division=0)
    return {
        "手法": name,
        "正解率": round(accuracy_score(y_true, y_pred), 3),
        "SH適合率": round(rep["1"]["precision"], 3),
        "SH再現率": round(rep["1"]["recall"], 3),
        "SH F1": round(rep["1"]["f1-score"], 3),
        "macro F1": round(rep["macro avg"]["f1-score"], 3),
    }
```

```
all_results = [
    get_metrics(y_test, y_pred_dt, "決定木(DT)"),
    get_metrics(y_test, y_pred_rf, "ランダムフォレスト(RF)"),
    get_metrics(y_test, y_pred_xgb, "XGBoost"),
    get_metrics(y_test, y_pred_lr, "ロジスティック回帰(LR)"),
    get_metrics(y_test, y_pred_svm, "SVM(線形)"),
]
```

```
if has_xgb:
```

```
    all_results.insert(2, get_metrics(y_test, y_pred_xgb, "XGBoost"))
```

```
summary_df = pd.DataFrame(all_results).set_index("手法")
print(summary_df.to_string())
```

#次に続く。。

手法	正解率	SH適合率	SH再現率	SH F1	macro F1
決定木 (DT)	0.725	0.703	0.658	0.680	0.720
ランダムフォレスト (RF)	0.755	0.735	0.702	0.718	0.751
XGBoost	0.751	0.732	0.693	0.712	0.747
ロジスティック回帰 (LR)	0.688	0.649	0.648	0.648	0.684
SVM (線形)	0.697	0.639	0.733	0.682	0.697

第5～7回の全手法比較

いろいろ試してみよう！

— 棒グラフで比較 —

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
metrics_pairs = [    ["正解率", "SH再現率"],    ["SH F1",  "macro F1"],]
pal = ["#0F2B46", "#27AE60", "#E67E22", "#7D3C98", "#C0392B"]

x = np.arange(len(summary_df))
w = 0.38

for ax, (m1, m2) in zip(axes, metrics_pairs):
    b1 = ax.bar(x - w/2, summary_df[m1], w, label=m1, color=pal[0], alpha=0.85)
    b2 = ax.bar(x + w/2, summary_df[m2], w, label=m2, color=pal[1], alpha=0.85)
    for b in [*b1, *b2]:
        ax.text(b.get_x()+b.get_width()/2, b.get_height()+0.005,
                f"{b.get_height():.3f}", ha="center", va="bottom",
                fontsize=8.5)
    ax.set_xticks(x)
    ax.set_xticklabels(summary_df.index, fontsize=9, rotation=15, ha="right")
    ax.set_ylim(0.4, 1.0)
    ax.legend(fontsize=10)
    ax.grid(True, axis="y", alpha=0.3)
    ax.set_title(f"{m1} vs {m2}", fontsize=12)

plt.suptitle("第5～7回 全手法の精度比較(Airbnb Tokyo:スーパーホスト判定)", fontsize=12, fontweight="bold")
plt.tight_layout()
plt.savefig("all_methods_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```

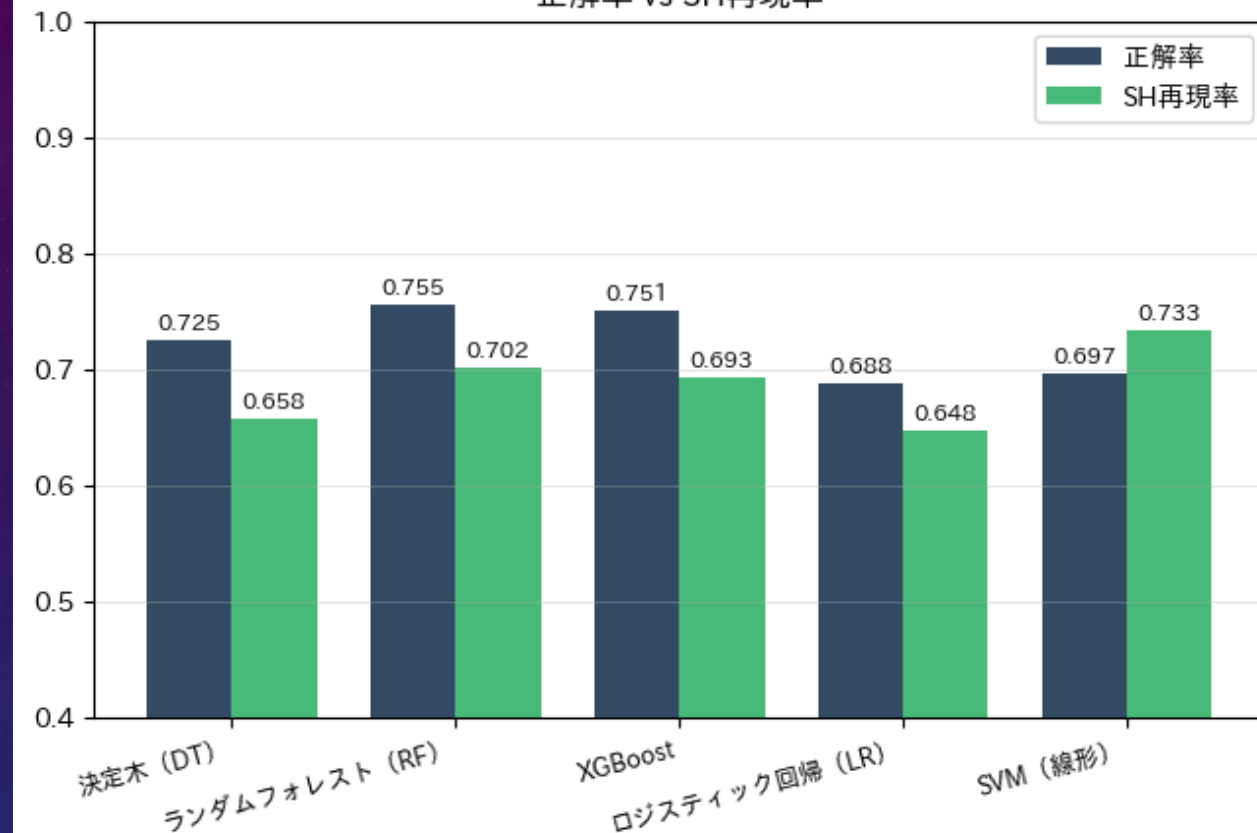
#次に続く。。

第5～7回の全手法比較

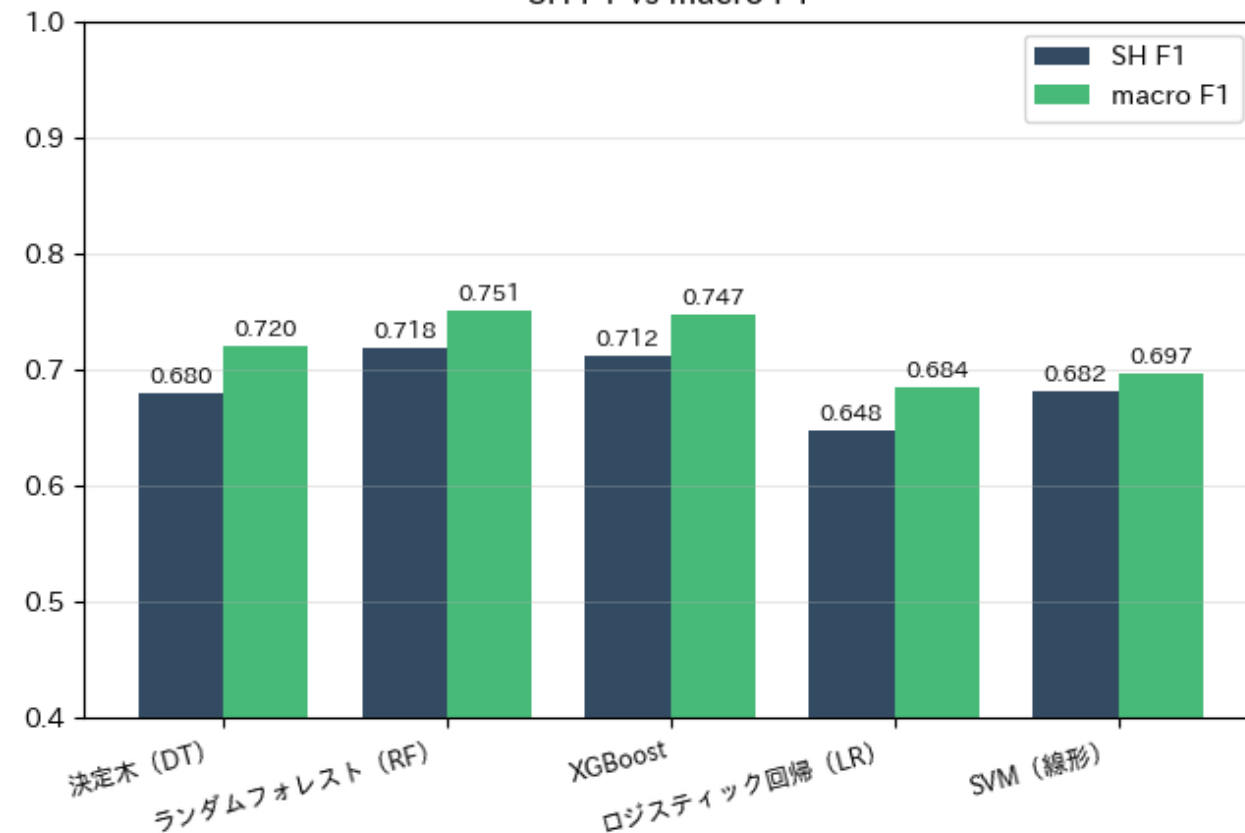
いろいろ試してみよう！

第5～7回 全手法の精度比較 (Airbnb Tokyo : スーパーホスト判定)

正解率 vs SH再現率



SH F1 vs macro F1



第5～7回の全手法比較

いろいろ試してみよう！

ベースラインとの比較

```
baseline = (y_test == 0).mean()
```

```
print(f"¥nベースライン(全員を非SHと予測)の正解率:{baseline:.3f}")
```

ベースライン（全員を非SHと予測）の正解率：0.556

ROC曲線の比較

いろいろ試してみよう！

ROC曲線の比較

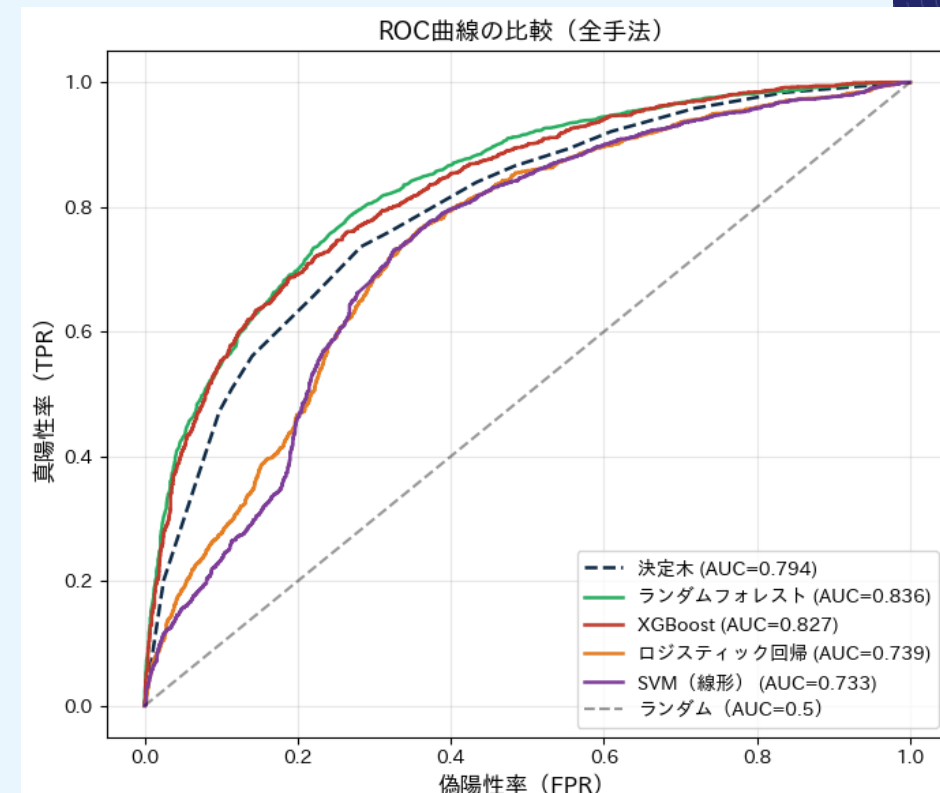
```
fig, ax = plt.subplots(figsize=(7, 6))
roc_models = [
    ("決定木", dt, X_test, "#0F2B46", "--"),
    ("ランダムフォレスト", rf, X_test, "#27AE60", "-"),
    ("XGBoost", xgb_model, X_test, "#C0392B", "-"),
    ("ロジスティック回帰", lr, X_test_sc, "#E67E22", "-"),
    ("SVM(線形)", svm, X_test_sc, "#7D3C98", "-"),
]
```

```
if has_xgb:
```

```
roc_models.insert(2, ("XGBoost", xgb_model, X_test, "#C0392B", "-"))
```

```
for name, model, X_inp, color, ls in roc_models:
    proba = model.predict_proba(X_inp)[ :, 1]
    fpr, tpr, _ = roc_curve(y_test, proba)
    roc_auc = auc(fpr, tpr)
    ax.plot(fpr, tpr, color=color, linestyle=ls, linewidth=1.8,
            label=f"{name} (AUC={roc_auc:.3f})")
```

```
ax.plot([0, 1], [0, 1], "k--", alpha=0.4, label="ランダム(AUC=0.5)")
ax.set_xlabel("偽陽性率(FPR)", fontsize=12)
ax.set_ylabel("真陽性率(TPR)", fontsize=12)
ax.set_title("ROC曲線の比較(全手法)", fontsize=13)
ax.legend(fontsize=10, loc="lower right")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("roc_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```



3

本日の課題

頑張ってください。



課題

【課題】 期限：2026/6/3（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第7回（5/27）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```

【ADSセミナー】

本学の「先端データ科学研究センター」が主催する
セミナーが6/25（木）から4週にわたって
開催されます。

内容が盛りだくさんなので、**研究室を選択する前の
参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

ADSセミナー

対象 東京情報大学
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生
総合情報学科 准教授（数理情報研究ユニット）



量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生
総合情報学科 准教授（情報セキュリティ研究ユニット）



個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となる場合があります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生
総合情報学科 教授（情報メディア学系）



音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生
総合情報学科 准教授（生命情報研究ユニット）



身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

<参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時
募集中

<お問い合わせ>

東京情報大学 総合情報学部
先端データ科学研究センター
ads@rsch.tuis.ac.jp

